

The EFGP-SING E-step

GP regression on uncertain latent velocities, solved by Fourier features

Abstract

This note derives the E-step of SING with a GP drift prior, accelerated by EFGP, organised around *which ELBO term each step addresses*. The spine is GP regression on latent velocities. Three layers of structure turn it into a matrix-free linear system: replacing fixed sufficient statistics by $q(x)$ -expectations and applying Stein’s lemma; choosing Fourier features so the system matrix is BTTB Toeplitz with a cheap Jacobian; and computing the remaining $q(x)$ -expectations in closed form via the Gaussian characteristic function. The $q(f) \rightarrow q(x)$ handoff is closed by one principle — a first-order Taylor linearisation at m_i — applied to two pieces of the transition integrand and yielding PSD-by-construction natural-grad updates.

Contents

1	Setup and ELBO road-map	2
2	Notation cheat-sheet	3
3	The $q(f)$ update: GP regression on latent velocities	3
3.1	Transition stripped to its f -content	4
3.2	Layer 1: GP regression with known x	4
3.3	Layer 2: uncertain x , in a finite basis	4
4	The EFGP basis choice	5
5	Computing the $q(x)$-expectations that build A_r, h_r	6
5.1	Closed-form via the Gaussian characteristic function (default)	6
5.2	Monte Carlo cloud (reference)	6
6	The $q(x)$ natural-gradient step	7
7	Algorithm summary and complexity	13
A	Math-to-code dictionary	14
B	V-Hutch restoration (opt-in)	14
B.1	Closed-form $\mathbb{E}_{q(x_i)}[V]$ via BTTB-diagonal sums of $DA^{-1}D$	15
B.2	Hutchinson estimator of ω_r	15
B.3	Per-transition S_i -gradient via Taylor	15
B.4	Homogeneous- S vs heterogeneous- S sub-modes	15

C Implementation and benchmarks	16
C.1 Custom-VJP shims: why a shim, not pure autodiff	16
C.2 Point-eval alternative: linearised-shim	16
C.3 Wall-time comparisons ($K = 10, T = 500$)	16
C.4 ∇V magnitude diagnostic	17
C.5 Op count for V-Hutch per outer iter	17

1 Setup and ELBO road-map

Latent SDE and observations

We assume an Euler-discretised latent SDE

$$x_{i+1} = x_i + \Delta_i f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \Delta_i \Sigma), \quad (1)$$

with Gaussian observations $p(y_i | x_i)$. Throughout, we abbreviate

$$\delta_i := x_{i+1} - x_i. \quad (2)$$

The diffusion is taken diagonal, $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_{D_x}^2)$. The drift coordinates carry independent GP priors,

$$f_r \sim \text{GP}(0, k_\theta), \quad r = 1, \dots, D_x. \quad (3)$$

Joint, ELBO, and the three vEM updates

The joint factorises as

$$p(f, x_{0:T}, y_{0:T}) = p(f) p(x_0) \prod_i p(x_{i+1} | x_i, f) \prod_i p(y_i | x_i), \quad (4)$$

with structured variational posterior $q(x_{0:T}) q(f)$. The ELBO is

$$\begin{aligned} \mathcal{L}(q) = & \underbrace{\mathbb{E}_{q(x)}[\log p(y | x)]}_{\substack{\text{Gaussian closed-form;} \\ q(x)\text{-grad: backbone autograd}}} + \underbrace{\sum_i \mathbb{E}_{q(x,f)}[\log p(x_{i+1} | x_i, f)]}_{\substack{\text{transition — couples } q(x), q(f); \\ q(x)\text{-grad: } \textit{hand-derived custom_vjp} \text{ (§6); } \\ q(f)\text{-update: closed-form } A_r \mu_r = h_r \text{ (§3)}}} \\ & + \underbrace{\mathbb{E}_{q(f)}[\log p(f) - \log q(f)]}_{\substack{\text{constant in } q(x); \\ \text{enters } q(f)\text{-update + M-step}}} + \underbrace{H[q(x)] + \mathbb{E}_{q(x_0)}[\log p(x_0)]}_{\substack{\text{Gaussian closed-form;} \\ q(x)\text{-grad: backbone autograd}}}. \end{aligned} \quad (5)$$

vEM cycles three coupled updates against this ELBO:

- q(f) update** (§3–§5): maximise $\mathcal{L}$ over $q(f)$ with $q(x)$ fixed. Closed-form Gaussian $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$; $A_r \mu_r = h_r$ via Jacobi-PCG.
- q(x) update** (§6): SING ρ -damped natural gradient on $\mathcal{L}$ with $q(f)$ fixed. The transition contribution's $q(x)$ -integrals are closed in x by Stein (for the pairwise piece) and a first-order Taylor linearisation at m_i (for the residual nonlinearity in $\tilde{f}$ and V).

M-step (separate doc): kernel hypers θ maximise the collapsed marginal NLL. At the $q(f)$ -optimum,

$$\mathcal{L}_M(\theta) = -\frac{1}{2} \sum_r \text{Re} \langle h_{\theta,r}, \mu_{\theta,r} \rangle + \frac{D_{\text{out}}}{2} \log |A_\theta|. \quad (6)$$

Only the transition term in (5) couples $q(x)$ and $q(f)$ non-trivially; the rest of this document analyses that term.

Section map

§	Addresses	Produces
2	—	Notation glossary
3 (incl. 3.1–3.3)	transition at fixed $q(x)$	A_r, h_r (basis-agnostic)
4	same	Fourier specialisation: BTTB + cheap Jacobian
5	same	the $q(x)$ -expectations that fill A_r, h_r
6 (incl. 6.6)	transition at fixed $q(f)$	per-transition $(\partial/\partial m_i, \partial/\partial S_i, \partial/\partial S_{i,i+1})$ via Stein + Taylor at $m_i +$
7	—	end-to-end pseudocode + complexity

Teaser: the two updates in one breath

q(f) update. From $q(x)$ marginals build A_r, h_r via NUFFT primitives over the $q(x_i) = \mathcal{N}(m_i, S_i)$ Gaussians; solve $A_r \mu_r = h_r$ by Jacobi-preconditioned CG.

q(x) update. From μ_r , get $(\bar{f}, J_{\bar{f}})$ at each m_i via Type-2 NUFFT; substitute their Taylor-linearised form into the transition expectation; differentiate w.r.t. (m_i, S_i) via `jax.grad` (custom-VJP shim plumbs the gradient table from §6).

2 Notation cheat-sheet

m_i, S_i	marginal mean and covariance of $q(x_i)$.
$S_{i,i+1}$	cross-covariance $\text{Cov}(x_i, x_{i+1})$.
$d_i := m_{i+1} - m_i$	mean increment.
$C_i := S_{i,i+1} - S_i = \text{Cov}(x_i, x_{i+1} - x_i)$	Stein cross-cov correction.
$\phi(x), \phi_\theta(x)$	finite basis for f ; subscript θ added when the kernel parameterisation matters (§4 onward).
ξ_k, D_θ	equispaced Fourier spectral grid and diagonal kernel-weight matrix (eq. 17).
A_r, h_r, μ_r	$q(f_r)$ precision, RHS, and posterior mean in the basis (eq. 17).
$\bar{f}_r(x) = \phi(x)^* \mu_r$	posterior mean drift under $q(f)$ (exact).
$\tilde{f}(x_i)$	first-order Taylor of $\bar{f}$ at m_i , the §6 linearisation.
$J_{\bar{f}}(x)$	Jacobian of $\bar{f}$ at x ; for Fourier features cheap (eq. 21).
$V_r(x) = \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$	$q(f)$ posterior variance contribution at x (eq. 38).
$\omega_r(\delta)$	BTTB-diagonal projection of $D_\theta A_r^{-1} D_\theta$ on lag δ (App. B, eq. 42).

3 The q(f) update: GP regression on latent velocities

ELBO term: transition term $\sum_i \mathbb{E}_{q(x,f)} [\log p(x_{i+1} | x_i, f)] - \text{KL}[q(f) \| p(f)]$ at fixed $q(x)$. **Quantity produced:** closed-form Gaussian $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$ via $A_r \mu_r = h_r$ (eq. 17).

3.1 Transition stripped to its f -content

The Euler transition log-density is

$$\log p(x_{i+1} | x_i, f) = -\frac{1}{2\Delta_i} [\delta_i - \Delta_i f(x_i)]^\top \Sigma^{-1} [\delta_i - \Delta_i f(x_i)] + \text{const.} \quad (7)$$

The constant and the $\delta_i^\top \Sigma^{-1} \delta_i$ term do not involve f . Expanding for diagonal Σ , the part of $\log p(x_{i+1} | x_i, f)$ that depends on coordinate f_r is

$$\boxed{\frac{1}{\sigma_r^2} \delta_{i,r} f_r(x_i) - \frac{\Delta_i}{2\sigma_r^2} f_r(x_i)^2.} \quad (8)$$

This is a Gaussian likelihood, quadratic in the function values $f_r(x_i)$. That is the *only* fact about the transition we will need. The $q(f_r)$ update reduces to combining (8) with the GP prior (3). Because Σ is diagonal and the f_r -priors are independent, the $q(f_r)$ updates decouple across r ; from now on we work coordinate-wise and suppress the index r when convenient.

3.2 Layer 1: GP regression with known x

Pretend, for this section only, that the latent path $X = \{x_0, \dots, x_{T-1}\}$ is observed. Define the pseudo-velocity

$$y_{i,r}^{\text{pseudo}} := \frac{\delta_{i,r}}{\Delta_i} = \frac{x_{i+1,r} - x_{i,r}}{\Delta_i}. \quad (9)$$

Completing the square in (8) shows the contribution from transition i is

$$-\frac{\Delta_i}{2\sigma_r^2} (y_{i,r}^{\text{pseudo}} - f_r(x_i))^2 + \text{const}, \quad (10)$$

i.e. a Gaussian observation of $f_r(x_i)$ with precision $\lambda_{i,r} = \Delta_i/\sigma_r^2$. Stack

$$f_{r,X} = [f_r(x_0), \dots, f_r(x_{T-1})]^\top, \quad [K_X]_{ij} = k_\theta(x_i, x_j), \quad \Lambda_r = \text{diag}(\lambda_{i,r}), \quad (11)$$

and let y_r^{pseudo} collect the pseudo-velocities. Then $q(f_{r,X}) = \mathcal{N}(m_{r,X}, C_{r,X})$ with the standard GP-regression identities

$$\boxed{C_{r,X}^{-1} = K_X^{-1} + \Lambda_r, \quad C_{r,X}^{-1} m_{r,X} = \Lambda_r y_r^{\text{pseudo}}.} \quad (12)$$

Template. The $q(f)$ update is GP regression on latent velocities. The remainder of this section modifies (12) to the case where x is uncertain; §4 then specialises the basis to make it FFT-cheap.

3.3 Layer 2: uncertain x , in a finite basis

In SING X is not observed; only $q(x_{0:T})$ is available, a Gaussian Markov variational posterior with marginals (m_i, S_i) and cross-covariances $S_{i,i+1}$. The K-space packaging (12) no longer applies directly — with X random, both $f_{r,X}$ and K_X are random — so rather than carry a function posterior we parametrise the drift directly by weights,

$$f_r(x) = \phi(x)^* w_r, \quad w_r \sim \mathcal{N}(0, I_M), \quad (13)$$

the basis chosen so the iid prior on w_r reproduces the GP prior on f_r . The coordinate-ascent VI update for $q(w_r)$ is then *just the expected log-joint as a function of w_r* :

$$\log q^*(w_r) = \mathbb{E}_{q(x)}[\log p(x_{0:T}, w_r)] + \text{const}, \quad (14)$$

the expectation over x only, with w_r the variable being updated. The only w_r -dependent pieces are the prior $\log p(w_r) = -\frac{1}{2} w_r^* w_r$ and the transition content (8) with $f_r(x_i) = \phi(x_i)^* w_r$ — so $\delta_{i,r} f_r(x_i) = \delta_{i,r} \phi(x_i)^* w_r$ is linear in w_r and $f_r(x_i)^2 = w_r^* \phi(x_i) \phi(x_i)^* w_r$ is quadratic. Hence

$$\log q^*(w_r) = -\frac{1}{2} w_r^* A_r w_r + \Re(h_r^* w_r) + \text{const}, \quad (15)$$

with

$$\boxed{A_r = I + \frac{1}{\sigma_r^2} \sum_i \Delta_i \mathbb{E}_{q(x_i)}[\phi(x_i) \phi(x_i)^*], \quad h_r = \frac{1}{\sigma_r^2} \sum_i \mathbb{E}_{q(x_i, x_{i+1})}[\phi(x_i) (x_{i+1,r} - x_{i,r})].} \quad (16)$$

The expectation over $q(x)$ touches only the features, contracting $\phi\phi^*$ and ϕ to the deterministic $M \times M$ matrix and M -vector in (16); w_r is never integrated. So $\log q^*(w_r)$ is exactly quadratic in w_r , and completing the square gives the Gaussian

$$q(w_r) = \mathcal{N}(\mu_r, A_r^{-1}), \quad A_r \mu_r = h_r. \quad (17)$$

Stein collapse of the linear term in h_r . The pairwise expectation in (16) under $q(x_i, x_{i+1})$ is awkward. Stein’s lemma collapses it to marginal expectations plus a cross-cov correction. With $d_i := m_{i+1} - m_i$ and $C_i := S_{i,i+1} - S_i = \text{Cov}(x_i, x_{i+1} - x_i)$, for any smooth g ,

$$\mathbb{E}[g(x_i) (x_{i+1,r} - x_{i,r})] = \mathbb{E}[g(x_i)] d_{i,r} + \mathbb{E}[J_g(x_i)] C_{i,r}. \quad (18)$$

Apply with $g = \phi$ (componentwise) inside (16):

$$\boxed{h_r = \frac{1}{\sigma_r^2} \sum_i \left\{ \underbrace{\mathbb{E}_{q(x_i)}[\phi(x_i)] d_{i,r}}_{\text{mean-increment}} + \underbrace{\mathbb{E}_{q(x_i)}[J_\phi(x_i)]^* C_{i,r}}_{\text{Stein correction}} \right\}.} \quad (19)$$

Every quantity in (17) now depends only on *marginal* expectations under $q(x_i)$, plus the cross-cov correction C_i .

4 The EFGP basis choice

ELBO term: still the transition term at fixed $q(x)$. **Quantity produced:** Fourier basis specialisation of (17): A_r becomes BTTB-Toeplitz and J_ϕ is frequency multiplication.

EFGP specialises the basis ϕ in (13) to Fourier features:

$$\phi_\theta(x) = D_\theta F_x, \quad F_x[k] = e^{2\pi i x^\top \xi_k}, \quad \phi_\theta(x)^* \phi_\theta(x') \approx k_\theta(x, x'), \quad (20)$$

with $\{\xi_k\}_{k=1}^M$ an equispaced spectral grid and D_θ diagonal with square-root spectral weights chosen to match k_θ . This is the only kernel approximation introduced. The system (17) is unchanged in form; what we gain are two structural simplifications.

Cheap Jacobian. Differentiating (20),

$$\frac{\partial \phi_k(x)}{\partial x_j} = 2\pi i \xi_{k,j} \phi_k(x), \quad (21)$$

so J_ϕ is just frequency-multiplication of ϕ . The Stein correction in (19) costs no more than evaluating ϕ itself.

BTTB structure of $\mathbb{E}_{q(x_i)}[\phi\phi^*]$. For Fourier features, $\phi(x)\phi(x)^*$ has (k, ℓ) entry $D_{\theta,k} e^{2\pi i x^\top (\xi_\ell - \xi_k)} D_{\theta,\ell}$. The expectation under $q(x_i)$ depends on $\xi_\ell - \xi_k$ only:

$$(\mathbb{E}_{q(x_i)}[\phi\phi^*])_{k\ell} = D_{\theta,k} \mathbb{E}_{q(x_i)}[e^{2\pi i x_i^\top (\xi_\ell - \xi_k)}] D_{\theta,\ell}. \quad (22)$$

On an equispaced grid this makes $\mathbb{E}_{q(x_i)}[\phi\phi^*]$ Block-Toeplitz with Toeplitz Blocks (BTTB), so $A_r v$ costs $O(M \log M)$ via FFT instead of $O(M^2)$.

5 Computing the $q(x)$ -expectations that build A_r, h_r

ELBO term: still the transition term at fixed $q(x)$. **Quantity produced:** the marginal expectations $\mathbb{E}_{q(x_i)}[\phi(x_i)\phi(x_i)^*]$, $\mathbb{E}_{q(x_i)}[\phi(x_i)]$, $\mathbb{E}_{q(x_i)}[J_\phi(x_i)]$ that fill (17)–(19).

5.1 Closed-form via the Gaussian characteristic function (default)

Because every $q(x_i) = \mathcal{N}(m_i, S_i)$ is Gaussian and every ϕ_k is a complex exponential, each marginal expectation appearing in (16)–(19) is the Gaussian characteristic function evaluated at frequency differences:

$$\mathbb{E}_{q(x_i)}[e^{2\pi i x_i^\top \xi}] = e^{2\pi i m_i^\top \xi} e^{-2\pi^2 \xi^\top S_i \xi}, \quad (23)$$

i.e. a phase at m_i times a Gaussian envelope set by S_i . Substituting into A_r (16) preserves BTTB structure: the generating kernel on the difference grid $\Delta \xi_k$ is the Type-1 NUFFT of weighted Gaussians *centred at m_i with per-source widths S_i* . Operationally this replaces the fixed NUFFT spreading kernel by a per-source $\mathcal{N}(\cdot; m_i, S_i)$ on a truncated stencil; the FFT and deconvolve steps are unchanged. The RHS terms in (19) receive the same treatment.

The companion note `efgp_estep_charfun.tex` carries the full derivation, the explicit closed-form generator, complexity proofs, and the bucketed / Gauss–Hermite alternatives considered. The closed-form spreader eliminates Monte Carlo variance at the cost of a per-source spread; in single-scale regimes the asymptotic cost matches a standard NUFFT, and heterogeneous S_i costs an extra $\propto (\sigma_{\max}/\sigma_{\min})^D$ factor.

5.2 Monte Carlo cloud (reference)

The same expectations can be evaluated by Monte Carlo. For each $i = 0, \dots, T-2$, draw S marginal samples $x_i^{(s)} \sim q(x_i)$ and stack as $X = \{x_i^{(s)}\}_{i,s} \in \mathbb{R}^{(T-1)S \times D_x}$. With

$$[F_X]_{k,(i,s)} = e^{2\pi i (x_i^{(s)})^\top \xi_k}, \quad \phi_\theta(x_i^{(s)}) = D_\theta(F_X)_{\cdot,(i,s)}, \quad (24)$$

and per-sample weights $(W_r)_{(i,s)} = \Delta_i / (S\sigma_r^2)$, A_r becomes

$$A_r \approx I + D_\theta T_r D_\theta, \quad T_r := F_X^* W_r F_X, \quad (25)$$

with $(T_r)_{k\ell}$ depending on $\xi_\ell - \xi_k$ only. T_r is BTTB on the equispaced grid; a single Type-1 NUFFT over X produces its generating kernel, and $A_r v = v + D_\theta (T_r (D_\theta v))$. The system $A_r \mu_r = h_r$ is solved by Jacobi-preconditioned CG.

For h_r , set

$$(a_r)_{(i,s)} = \frac{d_{i,r}}{S \sigma_r^2}, \quad (c_{j,r})_{(i,s)} = \frac{[C_i]_{j,r}}{S \sigma_r^2}, \quad (26)$$

so (19) estimates as

$$h_{1,r} \approx D_\theta F_X^* a_r, \quad h_{2,r} \approx \sum_{j=1}^{D_x} (-2\pi i \xi_j) \odot D_\theta F_X^* c_{j,r}, \quad h_r = h_{1,r} + h_{2,r}. \quad (27)$$

The factor $-2\pi i \xi_j$ in $h_{2,r}$ (rather than $+2\pi i \xi_j$) comes from the adjoint Jacobian $J_\phi^* \cdot \overline{2\pi i \xi_{k,j} \phi_k(x)} = -2\pi i \xi_{k,j} \overline{\phi_k(x)}$.

The Stein form (19) can be checked against a naive pair-sampling estimator (draw transition pairs $(x_i^{(s)}, x_{i+1}^{(s)}) \sim q(x_i, x_{i+1})$ and use sampled pseudo-velocities in (12)); both estimators target the same h_r , with Stein strictly lower variance. SING uses Stein in production and pair-sampling only as a regression test.

6 The $q(x)$ natural-gradient step

ELBO term: transition term at fixed $q(f) - \sum_i \mathbb{E}_{q(x_i, x_{i+1})} [\mathbb{E}_{q(f)} [\log p(x_{i+1} | x_i, f)]]$. **Quantity produced:** per-transition $(\partial/\partial m_i, \partial/\partial S_i, \partial/\partial S_{i,i+1})$ of this term, ready for SING's ρ -damped natural-grad update on $q(x)$.

Setup

SING does ρ -damped natural gradient on $q(x)$ (constrained Gaussian-Markov). By exponential-family duality between $q(x)$'s natural parameters $\eta = (J, h, L)$ and mean parameters $\mu = (\mathbb{E}[x_i], \mathbb{E}[x_i x_i^\top], \mathbb{E}[x_i x_{i+1}^\top])$,

$$\tilde{\nabla}_\eta \mathcal{L} = \partial \mathcal{L} / \partial \mu. \quad (28)$$

SING ρ -damps and applies. Mean params re-parameterise cleanly to $(m_i, S_i, S_{i,i+1})$. Only the transition term couples $q(x)$ to $q(f)$, so the EFGP-specific work is the per-transition transition contribution; the other ELBO terms (emissions, $H[q(x)]$, initial) are Gaussian-closed-form in $q(x)$ and handled by SING's backbone.

The transition term, and how the code assembles it

At fixed $q(f)$, with $\bar{f}_r(x) = \phi(x)^* \mu_r$ deterministic but nonlinear in x and $V_r(x) = \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$ the $q(f)$ posterior variance contribution, the per-transition piece is

$$\mathbb{E}_{q(x_i, x_{i+1})} [\mathbb{E}_{q(f)} [\log p(x_{i+1} | x_i, f)]] = -\frac{\mathbb{E}[\delta_i^\top \Sigma^{-1} \delta_i]}{2\Delta_i} + \mathbb{E}_{q(x_i, x_{i+1})} [\delta_i^\top \Sigma^{-1} \bar{f}(x_i)] - \frac{\Delta_i}{2} \mathbb{E}_{q(x_i)} [\bar{f}^\top \Sigma^{-1} \bar{f} + V] + \text{const}. \quad (29)$$

`compute_neg_CE_single` (`sing/utils/sing_helpers.py:289-304`) assembles this into a scalar `trm` from four pieces:

piece	math	where <code>trm</code> is built (lines in <code>compute_neg_CE_single</code>)
(i)	$\mathbb{E}[\delta^\top \Sigma^{-1} \delta] / (2\Delta_i)$	294-296 (traces of m, S, SS)
(ii)	$\mathbb{E}_{q(x_i, x_{i+1})}[\delta^\top \Sigma^{-1} \bar{f}]$	298-299 (<code>Eff</code> and <code>Edfdx</code>)
(iii)	$\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$	297 (<code>((del.t)**2 * Eff)</code>)
(iv)	$\mathbb{E}_{q(x_i)}[V]$	dropped (production)

The drift moments `Eff`, `Eff`, `Edfdx` consumed in lines 297-299 are pre-computed off-graph by `drift_moments_gmix_jax` and exposed via the `FrozenEFGPDrift` pytree — from `compute_neg_CE_single`'s view they are per-transition constants. The Taylor framework below dictates how their (m_t, S_t) -gradients are filled in via `custom_vjp`; the four item subsections then walk each piece's value and gradient flow.

The Gauss-Newton framework

Items (ii)-(iv) need (m_i, S_i) -derivatives of expectations under $q(x_i) = \mathcal{N}(m_i, S_i)$ of nonlinear functions $(\bar{f}, J_{\bar{f}}, \bar{f}^\top \Sigma^{-1} \bar{f}, V)$. Production handles all of these under a single principled approximation: a Gauss-Newton (GN) truncation of Price's theorem.

Price's theorem (Bonnet's lemma). For any sufficiently smooth g and Gaussian $q(x_i)$,

$$\partial_{m_i} \mathbb{E}_{q(x_i)}[g(x_i)] = \mathbb{E}_{q(x_i)}[\nabla g(x_i)], \quad \partial_{S_i} \mathbb{E}_{q(x_i)}[g(x_i)] = \frac{1}{2} \mathbb{E}_{q(x_i)}[\nabla^2 g(x_i)]. \quad (30)$$

These are exact identities, expressing the (m, S) -derivatives of a Gaussian expectation as expectations of the gradient and Hessian of the integrand.

The GN move on the load-bearing slot. For $g = \bar{f}^\top \Sigma^{-1} \bar{f}$ (the quadratic in item (iii)), the Hessian splits as

$$\nabla^2(\bar{f}^\top \Sigma^{-1} \bar{f}) = 2 \left(\underbrace{J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}}_{\text{GN piece: PSD}} + \underbrace{\sum_r \sigma_r^{-2} \bar{f}_r H_{\bar{f}_r}}_{\text{residual: sign-indefinite}} \right). \quad (31)$$

The first piece is PSD-by-construction; the second (the residual Hessian) is sign-indefinite because $\bar{f}_r$ has no sign constraint. **The Gauss-Newton approximation drops the residual Hessian** — the standard move for sum-of-squares objectives. Plugged into Price:

$$\partial_{S_i} \mathbb{E}[\bar{f}^\top \Sigma^{-1} \bar{f}] \stackrel{\text{Price}}{=} \mathbb{E}[J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}] + \frac{1}{2} \mathbb{E}[\sum_r \sigma_r^{-2} \bar{f}_r H_{\bar{f}_r}] \stackrel{\text{GN}}{\approx} \mathbb{E}[J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}] \stackrel{\text{Cov-drop}}{\approx} \mathbb{E}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}[J_{\bar{f}}] \succeq 0. \quad (32)$$

The second approximation (the covariance drop) replaces $\mathbb{E}[J^\top J]$ with $\mathbb{E}[J]^\top \mathbb{E}[J]$ — also second-order in the nonlinearity of $\bar{f}$. The result is PSD-by-construction, a structural consequence of the GN form rather than a coincidence.

The same GN move handles every other truncation. For the linear pieces, Price + GN gives

$$\partial_{S_i} \mathbb{E}[\bar{f}] \stackrel{\text{Price}}{=} \frac{1}{2} \mathbb{E}[H_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0, \quad \partial_{m_i} \mathbb{E}[J_{\bar{f}}] \stackrel{\text{Price}}{=} \mathbb{E}[H_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0, \quad \partial_{S_i} \mathbb{E}[J_{\bar{f}}] \stackrel{\text{Price}}{=} \frac{1}{2} \mathbb{E}[\nabla^2 J_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0. \quad (33)$$

Every dropped slot is a residual Hessian (of $\bar{f}$ or of $J_{\bar{f}}$) — the same object GN drops in the quadratic case. The cross-term gradient $\partial_{m_i} \mathbb{E}[\bar{f}^\top \Sigma^{-1} \bar{f}] = 2 \mathbb{E}[J^\top \Sigma^{-1} \bar{f}]$ gets a further covariance drop in production: $2 \mathbb{E}[J]^\top \Sigma^{-1} \mathbb{E}[\bar{f}]$.

Gradient table (GN-truncated Price’s identities).

quantity	value	$\partial/\partial m_i$
$\mathbb{E}_{q(x_i)}[\bar{f}]$	$\mathbb{E}_{q(x_i)}[\bar{f}]$	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ 0 (GN: $\frac{1}{2}$)
$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$	0 (GN: $\mathbb{E}[H_{\bar{f}}]$ dropped)
$\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$	$\mathbb{E}_{q(x_i)}[\bar{f}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}] + \text{tr}[\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[J_{\bar{f}}] S_i]$	$2 \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]$ $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top$

Why this is justified. GN is a well-studied Hessian approximation for sum-of-squares objectives: drop the residual Hessian, keep the PSD outer-product piece. Exact when residuals vanish ($\bar{f}_r \rightarrow 0$); accurate when $\|\bar{f}_r\| \|H_{\bar{f}_r}\| \ll \|J_{\bar{f}}\|^2$. The covariance drops $\text{Cov}(J, J)$ and $\text{Cov}(J, \bar{f})$ are of the same order. This is the same approximation underlying variational Gauss-Newton (Khan & Lin, 2017) and drift-linearisation in variational SDE inference (Oppor & Archambeau, 2009).

Why we can’t just autodiff. The exact $\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$ closes via spectral autocorrelation of $\mu_r D\theta$,

$$\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}] = \sum_r \sigma_r^{-2} \sum_{k,\ell} \mu_{r,k}^* D_k D_\ell \mu_{r,\ell} e^{2\pi i m_i^\top \delta_{k\ell}} e^{-2\pi^2 \delta_{k\ell}^\top S_i \delta_{k\ell}}, \quad \delta_{k\ell} := \xi_\ell - \xi_k, \quad (34)$$

but ∂_{S_i} of this is a sign-indefinite sum of $\delta\delta^\top$ matrices (the full Hessian contribution to Price) — it would break the PSD property SING’s J -update relies on. The GN-truncated form (32) is what gives the load-bearing $\partial_{S_i} \geq 0$. Separately, `jax.grad` through the spectral closed forms costs $\sim 50\times$ per inner iter under `vmap(grad(...))` (App. C).

Production base values via gmix-gather. The expectations $\mathbb{E}_{q(x_i)}[\bar{f}]$ and $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ in the table slots are computed exactly in closed form via the Gaussian char-fun (23): with $\bar{f}_r(x) = \phi(x)^* \mu_r$,

$$\mathbb{E}_{q(x_i)}[\bar{f}_r] = \sum_k \mu_{r,k} D_{\theta,k} e^{2\pi i m_i^\top \xi_k} e^{-2\pi^2 \xi_k^\top S_i \xi_k},$$

analogously $\mathbb{E}_{q(x_i)}[\partial_j \bar{f}_r]$ with frequency modulation by $(2\pi i \xi_{k,j})$. The `gmix-gather` primitive in `drift_moments_gmix_jax` (`efgp_jax_drift.py:557-624`; Type-2 analog of the §5.1 spreader: one batched IFFT + per-source Gaussian stencil gather) delivers (`Ef`, `Eff`, `Edfdx`) at all T base points in one shot.

Implementation: GN-as-custom_vjp. The GN choice maps operationally to *treat* $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ as constant w.r.t. (m_i, S_i) in `jax.grad`’s chain rule. Implemented in `FrozenEFGPDrift` via three mechanisms:

- `Edfdx` ($= \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$) exposed as a raw array, *no custom_vjp*; chain rule through `Edfdx` returns 0. This implements the GN drops $\partial_m \mathbb{E}[J_{\bar{f}}] = \mathbb{E}[H_{\bar{f}}] \rightarrow 0$ and $\partial_S \mathbb{E}[J_{\bar{f}}] \rightarrow 0$.
- `Ef` ($= \mathbb{E}_{q(x_i)}[\bar{f}]$) through `ef_with_jac_grad`’s `custom_vjp`: backward returns $\partial \text{Ef} / \partial m = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top g$, $\partial \text{Ef} / \partial S = 0$. The ∂_m injection coincides with the exact Price value $\partial_m \mathbb{E}_{q(x_i)}[\bar{f}] = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ (Price reduces to a single gradient term here, no Hessian to drop); the $\partial_S = 0$ drops $\frac{1}{2} \mathbb{E}[H_{\bar{f}}]$, the GN move.

- **Eff** ($= \|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$) through `eff.with_grads`'s `custom_vjp`: backward returns the GN entries directly, $\partial_m = 2g(\mathbb{E}_{q(x_i)}[\bar{f}] \cdot \mathbb{E}_{q(x_i)}[J_{\bar{f}}])$ and $\partial_S = g \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$. The forward stores only $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$; the tr-piece is absorbed into the gradient (saves a per-source $D \times D$ contraction, since SING uses the gradient not the value).

Strict-Taylor as a special case. Replacing $\bar{f}$ by its first-order Taylor expansion at m_i ,

$$\tilde{f}(x_i) := \bar{f}(m_i) + J_{\bar{f}}(m_i)(x_i - m_i), \quad (35)$$

with $J_{\bar{f}}(m_i)$ treated as constant w.r.t. (m_i, S_i) , gives the same gradient table by a different route. The two derivations are equivalent; the GN derivation is cleaner because (a) it doesn't introduce a surrogate, (b) the PSD property of the load-bearing slot is structural, and (c) it connects to standard variational-GN literature.

Item (i): the δ -quadratic — exact

Math. $\mathbb{E}[\delta^\top \Sigma^{-1} \delta] / (2\Delta_i)$, a Gaussian quadratic in $\delta = x_{i+1} - x_i$ under $q(x_i, x_{i+1})$ — closed-form in $(m_i, m_{i+1}, S_i, S_{i+1}, S_{i,i+1})$.

Value (code, lines 294-296).

```
trm = trace(St_next + outer(mt_next, mt_next))
trm += trace(St + outer(mt, mt))
trm += -2 * trace(SS + outer(mt_next, mt))
```

Gradients. Pure `jax.grad` autodiff. No `custom_vjp`. SING-backbone, not EFGP-specific.

Item (ii): the Stein cross-term — value exact; gradients mix exact and GN-truncated pieces

Math. The Stein identity from §3.3, applied with $g = \bar{f}$, collapses the pairwise expectation *exactly* into a mean-increment piece and a Jacobian (Stein-correction) piece:

$$\mathbb{E}_{q(x_i, x_{i+1})}[\delta_i^\top \Sigma^{-1} \bar{f}(x_i)] = \underbrace{d_i^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]}_{\text{Part A: mean-increment}} + \underbrace{\text{tr}[\Sigma^{-1} \mathbb{E}_{q(x_i)}[J_{\bar{f}}] C_i^\top]}_{\text{Part B: Jacobian}}, \quad d_i := m_{i+1} - m_i, \quad C_i := S_{i,i+1} - S_i. \quad (36)$$

The marginals $\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ are computed by `gmix-gather` in closed form (framework above) and stored as `Ef`, `Edfdx` on `FrozenEFGPDrift`.

Value (code, lines 298-299).

```
trm += -2 * del_t * trace(outer(Ef, mt_next) + Edfdx @ SS.T)
trm += 2 * del_t * trace(outer(Ef, mt) + Edfdx @ St)
```

The outer-product terms are Part A in code form (combining to $-2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]^\top d_i$); the `Edfdx @` ... terms are Part B ($-2\Delta_i \text{tr}(\mathbb{E}_{q(x_i)}[J_{\bar{f}}] C_i^\top)$). The values themselves are exact.

Gradients. `jax.grad` of `trm` assembles three distinct contributions per $(\cdot)$ -slot. All production values and gradients are expressed in terms of the exact $\mathbb{E}_{q(x_i)}$ expectations $\mathbb{E}_{q(x_i)}[\bar{f}]$ and $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ from `gmix-gather`; the framework table's pointwise $\bar{f}(m_i)$, $J_{\bar{f}}(m_i)$ slots are filled with these smoothed values.

- **Direct.** Pure autodiff through the outer products and traces in lines 298-299 (treating `Ef` and `Edfdx` as constants). Exact.

- **Ef-chain.** Via `ef_with_jac_grad`'s `custom_vjp`, which injects $\partial \text{Ef} / \partial m = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top g$ and $\partial \text{Ef} / \partial S = 0$. The ∂_m injection *coincides exactly* with the true $\partial_m \mathbb{E}_{q(x_i)}[f] = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ from differentiating the char-fun term-by-term — no GN truncation here (Price reduces to a single gradient term; no Hessian to drop). The ∂_S injection *is* GN-truncated: by Price, $\partial_S \mathbb{E}[\bar{f}] = \frac{1}{2} \mathbb{E}[H_{\bar{f}}]$; GN drops the Hessian, giving 0 (exact would be the non-zero envelope-derivative $-2\pi^2 \sum_k \mu_{r,k} D_k \xi_k \xi_k^\top e^{2\pi i m^\top \xi_k} e^{-2\pi^2 \xi_k^\top S \xi_k}$).
- **Edfdx-chain.** `Edfdx` is a raw constant array on `FrozenEFGPDrift`; all $\partial \text{Edfdx} / \partial (m, S) = 0$. This is the framework's " $\mathbb{E}_{q(x_i)}[J]$ held constant" rule — the GN truncation drops $\mathbb{E}[H_{\bar{f}}]$ (∂_m) and the envelope-derivative (∂_S), both non-zero exactly.

Resolving per slot of the production gradient, each contribution labelled exact / GN-truncated:

slot	direct	Ef-chain	Edfdx-chain
∂m_t	$2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]$ (<i>exact</i>)	$-2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top d_i$ (<i>exact</i>)	0 (<i>GN: $\mathbb{E}[H_{\bar{f}}]$ dropped</i>)
∂m_{t+1}	$-2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]$ (<i>exact</i>)	0 (no m_{t+1} in Ef)	0 (Edfdx indep. of m_{t+1} ; exact)
∂S_t	$2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top$ (<i>exact, from $\text{tr}(\text{Edfdx } S_t)$</i>)	0 (<i>GN: env-deriv dropped</i>)	0 (<i>GN: env-deriv dropped</i>)
$\partial S_{i,i+1}$	$-2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top$ (<i>exact</i>)	0 (no $S_{i,i+1}$)	0 (Edfdx indep. of $S_{i,i+1}$; exact)

Net contributions of (ii) to ∂trm :

$$\begin{aligned}
\partial m_t &= 2\Delta_i (\mathbb{E}_{q(x_i)}[\bar{f}] - \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top d_i), & \partial m_{t+1} &= -2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}], \\
\partial S_t &= 2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top, & \partial S_{i,i+1} &= -2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top.
\end{aligned}$$

Status per slot: ∂m_{t+1} and $\partial S_{i,i+1}$ are *fully exact*; ∂m_t has Part A's contribution exact but Part B's GN-truncated ($\mathbb{E}[H_{\bar{f}}]$ dropped); ∂S_t has the direct piece exact but both chain contributions GN-truncated. Linear in $\bar{f}$; no PSD constraint applies (the PSD slot is item (iii)'s ∂S_t).

EFGP vs. SING. The mechanism that implements these GN drops — precomputing (Ef, Edfdx) off-graph via `gmix-gather` and exposing them with a `custom_vjp` that injects only the m -gradient on Ef — is EFGP-specific (the choice in `FrozenEFGPDrift`). SING's `nat_grad_transition` accepts whatever the SDE backend's (`f`, `ff`, `dfdx`) methods return. A different backend (e.g., one that integrates $\bar{f}$ against $q(x_i)$ on-graph via Gauss–Hermite quadrature) would let `jax.grad` propagate the exact $\partial_S \mathbb{E}_{q(x_i)}[\bar{f}]$ (the full Price contribution without the GN drop). EFGP picks the off-graph route to avoid autodiff through NUFFT's ($\sim 50\times$ cost under `vmap(grad(...)`); App. C); the GN framework's gradient table is what makes this trade exactly consistent (PSD-by-construction on the load-bearing slot, item (iii)).

Item (iii): the marginal quadratic — the GN slot

Math. $\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$. Apply Price's theorem and the GN move (framework above): the ∂_S derivative is the residual-Hessian decomposition (31), with the sign-indefinite residual piece dropped and the covariance further dropped, yielding (32). This is the load-bearing PSD-by-construction slot of the natural-grad. The exact spectral-autocorrelation form (34) is the alternative the GN framework rejected (sign-indefinite ∂_S).

Value (code, line 297). `trm += (del_t)**2 * Eff`, where `Eff` = $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$ is stored as `(Ef * Ef).sum(axis=-1)` in `drift_moments_gmix_jax` line 623. The forward holds only the $\mathbb{E}_{q(x_i)}[\bar{f}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]$ piece of (32); the tr-piece is absorbed into the gradient by the `custom_vjp` below (saves a per-source

contraction). The exact $\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$ via `gmix.E_full_Eff` (`efgp-jax-drift.py:88-102`) is separate, used for ELBO reporting only.

Gradients. `Eff` comes from `eff_with_grads(m, S, eff_const, ef_const, jac_const)` (`efgp-jax-drift.py:56-75`) the `custom_vjp` backward returns

$$\partial \text{Eff} / \partial m_t = 2g(\text{ef_const} \cdot \text{jac_const}), \quad \partial \text{Eff} / \partial S_t = g(\text{jac_const}^\top \text{jac_const}),$$

with g the upstream cotangent ($= (\Delta_i)^2$ from line 297), `ef_const` = $\mathbb{E}_{q(x_i)}[\bar{f}]$, `jac_const` = $\mathbb{E}_{q(x_i)}[J\bar{f}]$. Reading these against the GN-truncated Price's identities:

- ∂m_t injection $2g \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[\bar{f}]$. By Price, $\partial_m \mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}] = 2\mathbb{E}[J^\top \Sigma^{-1} \bar{f}]$; production drops $\text{Cov}(J, \bar{f})$ giving $2\mathbb{E}_{q(x_i)}[J]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]$. **Covariance drop** (one of the GN approximations).
- ∂S_t injection $g \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[J\bar{f}]$. By Price, $\partial_S \mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$ has the residual-Hessian decomposition (31); GN drops the residual + $\text{Cov}(J, J)$, leaving $\mathbb{E}_{q(x_i)}[J]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[J] \succeq 0$. **GN move on the load-bearing slot.**

Without the `custom_vjp`, autodiff would return zero in both slots (the forward `Eff` has no JAX-traced (m, S) -dependence). Net contributions of (iii) to ∂trm :

$$\partial m_t = 2(\Delta_i)^2 \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[\bar{f}], \quad \partial S_t = (\Delta_i)^2 \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[J\bar{f}] \succeq 0.$$

This is the GN slot: m -gradient from Part A' (exact slot), S -gradient from Part B' (Taylor's trace piece). The Part B' ∂S_t is the load-bearing PSD slot that protects SING's ρ -damped J -update.

Item (iv): the V term — production drops; App. B opt-in via Price + Hessian-drop

Math. $\mathbb{E}_{q(x_i)}[V(x_i)]$, where $V(x) = \sum_r \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$ is the $q(f)$ posterior variance contribution. From

$$\text{Cov}_{q(f)}[f_r(x), f_r(x')] = \phi(x)^* A_r^{-1} \phi(x'), \quad (37)$$

the full $q(f)$ second moment of f splits as

$$\mathbb{E}_{q(f)}[f(x)^\top \Sigma^{-1} f(x)] = \sum_r \sigma_r^{-2} [\bar{f}_r(x)^2 + V_r(x)]; \quad (38)$$

V is the piece that (iii)'s $\bar{f}^\top \Sigma^{-1} \bar{f}$ doesn't capture. V has no natural Part A/B split — it's a single integral.

Production: value and gradients = 0. The $-\frac{\Delta_i}{2} \mathbb{E}_{q(x_i)}[V(x_i)]$ contribution is omitted from `trm`. Geometric justification: translation-invariance of stationary kernels gives $\nabla V(x) \approx 0$ wherever the source set $\{m_i^{(k)}\}$ looks isotropic at scale ℓ around x ; for SDE-recovery with diverse ICs and ergodic mixing this holds at every interior m_i , so the dropped $-\frac{\Delta_i}{2} \mathbb{E}_{q(x_i)}[\nabla V]$ contribution to the $q(x)$ gradient is negligible. Fails at boundary/edge points and cold-start.

App. B opt-in: Price + Hessian-drop. Apply Price's theorem (30) to $g = V$:

$$\partial_{m_i} \mathbb{E}_{q(x_i)}[V] = \mathbb{E}[\nabla V], \quad \partial_{S_i} \mathbb{E}_{q(x_i)}[V] = \frac{1}{2} \mathbb{E}[\nabla^2 V], \quad (39)$$

and apply the same Hessian-drop the framework uses elsewhere ($\mathbb{E}[\nabla^2 V] \rightarrow 0$). This gives

$$\mathbb{E}_{q(x_i)}[V] = V(m_i), \quad \partial_{m_i} \mathbb{E}_{q(x_i)}[V] = \nabla V(m_i), \quad \partial_{S_i} \mathbb{E}_{q(x_i)}[V] = 0. \quad (40)$$

All three slots are **first-order surrogates**: each replaces an exact Price-expectation expression by its constant- x leading term (drop the residual Hessian and any envelope-derivative contributions), exactly as for the $\bar{f}$ slots in (ii). V doesn't have a sum-of-squares structure — the dropped piece is just $\mathbb{E}[\nabla^2 V]$ rather than the GN residual — but the structural move (drop the Hessian under Price) is the same. $V(m_i)$ and $\nabla V(m_i)$ are precomputed off-graph (App. B: Hutchinson estimator of ω_r + batched NUFFT-2 against the difference-grid coefficient block — the analog of gmix-gather for the V piece); `FrozenEFGPDriftWithVHutch`'s `custom_vjp` injects the table above.

End-to-end: from trm gradients to a natural-grad step

1. Pre-compute drift moments $(\text{Ef}, \text{Eff}, \text{Edfdx}) = (\mathbb{E}_{q(x_i)}[\bar{f}], \|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2, \mathbb{E}_{q(x_i)}[J_{\bar{f}}])$ at all transition base points via `drift_moments_gmix_jax`; stored on `FrozenEFGPDrift`.
2. `nat_grad_transition(sing/sing.py)` runs `vmap(grad(compute_neg_CE_single))` over transitions.
3. `compute_neg_CE_single` consumes *mean parameters* $\mu = (\mathbb{E}[x_i], \mathbb{E}[x_i x_i^\top], \mathbb{E}[x_i x_{i+1}^\top])$, internally runs `mean_to_marginal_params` to recover $(m_i, S_i, S_{i,i+1})$, and assembles `trm` from items (i)-(iii).
4. `jax.grad` chain-rules: (i) pure autodiff through traces; (ii) autodiff + `ef_with_jac_grad`'s injected $\partial \text{Ef} / \partial m$; (iii) entirely via `eff_with_grads`'s injected $\partial \text{Eff} / \partial (m, S)$. Returns $\partial \mathcal{L} / \partial \mu$ — which by (28) is the natural gradient on (J, h, L) . SING ρ -damps and applies.

7 Algorithm summary and complexity

One outer iteration of the E-step

Given current $q(x_{0:T})$ (marginals m_i, S_i and cross-covariances $S_{i,i+1}$), kernel hyperparameters θ , diffusion Σ , and spectral grid $\{\xi_k\}$ with weights D_θ :

1. **Increments.** $d_i = m_{i+1} - m_i$, $C_i = S_{i,i+1} - S_i$.
2. **Sources.** For `estep_method='gmix'` (default), the “sources” are the $q(x_i) = \mathcal{N}(m_i, S_i)$ Gaussians themselves. For `'mc'`, draw $x_i^{(s)} \sim q(x_i)$.
3. **Precision.** Build the BTTB generator T_r from weighted sources (closed-form spreader for gmix, Type-1 NUFFT for MC) with weights Δ_i / σ_r^2 ; A_r acts as $v \mapsto v + D_\theta T_r D_\theta v$.
4. **RHS.** Same primitive with weights $d_{i,r} / \sigma_r^2$ and $[C_i]_{j,r} / \sigma_r^2$ produces $h_{1,r}$ and $h_{2,r}$.
5. **Solve.** Jacobi-preconditioned CG: $A_r \mu_r = h_r$.
6. **Drift moments.** Compute $\mathbb{E}_{q(x_i)}[\bar{f}], \mathbb{E}_{q(x_i)}[J_{\bar{f}}], \|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$ at all base points via gmix-gather (`drift_moments_gmix_jax`; §6).
7. **$q(x)$ update.** Feed these into `compute_neg_CE_single` via `FrozenEFGPDrift`; `jax.grad` inside `nat_grad_transition` returns $\partial \mathcal{L} / \partial \mu$, which by (28) is the natural gradient on (J, h, L) . SING ρ -damps and applies.

Complexity

Let N be the source count ($N = K(T-1)$ for the gmix path, $N = K(T-1)S$ for MC; K is the number of trials), M the number of Fourier features, k the CG iteration count. The $q(f)$ update costs

$$\underbrace{O(N)}_{\text{spread / cloud assembly}} + \underbrace{O(M \log M)}_{\text{NUFFT / FFT}} + \underbrace{O(k D_x M \log M)}_{\text{CG over output dims}}. \quad (41)$$

Drift moments at T points cost $O(T) + O(D_x M \log M)$. The optional V -restoration of App. B adds $O(P D_x k M \log M)$ per outer iter for P Hutchinson probes plus $O((1 + D_x)(M \log M + N))$ for the $1+D_x$ batched NUFFTs that produce $(V_i, \nabla V_i)$; the per-inner-iter overhead is zero (the custom-VJP shim looks up precomputed values). The implementation rule is to never materialise K , Φ , $\Phi^* W \Phi$, or A_r^{-1} : every required action is a Fourier matvec, a BTTB matvec, or a CG solve.

A Math-to-code dictionary

Math	Code
Spectral grid weights $D_\theta = \text{diag}(\sqrt{S(\xi_k)} h^d)$	<code>grid.ws</code> (real part)
Frequency points ξ_k	<code>grid.xis_flat</code>
Adjoint Type-1 NUFFT F_X^*	<code>nufft1</code>
Forward Type-2 NUFFT $\phi(x)^* \mu$	<code>nufft2</code>
BTTB generator $T_r = F_X^* W_r F_X$	<code>bttb_conv_vec_weighted</code> $\rightarrow$ <code>make_toeplitz</code>
Operator $A_r v = v + D_\theta T_r D_\theta v$	<code>make_A_apply</code>
Solve $A_r \mu_r = h_r$ (Jacobi-preconditioned CG)	<code>cg_solve</code>
$h_r = h_{1,r} + h_{2,r}$ build	<code>compute_mu_r_jax</code>
$\bar{f}(x)$, $J_{\bar{f}}(x)$ pointwise (legacy)	<code>drift_moments_jax</code>
$\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$, $\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$ (gmix closed form)	<code>gmix_E_moments_at</code> (sing/efgp_gmix_qx_moments)
Spectral autocorrelation $\rho(\delta)$ precompute (one FFT/outer iter)	<code>precompute_aux</code>
Multi-trial flatten of $\{(m_i, S_i, d_i, C_i, \Delta_i)\}_{k,i}$	<code>_flatten_stein</code>
$\text{diag}(\Phi A_r^{-1} \Phi^*)$ (general Hutchinson primitive)	<code>hutchinson_diag</code>
$\omega_r(\delta)$ Hutchinson estimator (eq. 44)	<code>precompute_omega_per_r</code> (sing/efgp_qx_v_hutch)
$(V_i, \nabla V_i)$ batched NUFFT-2 precompute (eq. 43)	<code>precompute_V_and_grad_E_V_per_t</code>
V-Hutch custom-VJP shim into per-transition $q(x)$ update	<code>FrozenEFGPDriftWithVHutch</code>
Live gmix moments shim into $q(x)$ update (default)	<code>LiveGmixEFGPDrift</code>
Linearised custom-VJP shim into $q(x)$ update	<code>FrozenEFGPDrift</code> , <code>ef_with_jac_grad</code> , <code>eff_with_g</code>
$q(x)$ consumer	<code>nat_grad_transition</code> (sing/sing.py)

$q(x)$ statistic	Code
m_i (marginal mean)	<code>ms</code>
S_i (marginal covariance)	<code>Ss</code>
$\text{Cov}(x_i, x_{i+1}) = S_{i,i+1}$	<code>SSs.transpose(0,2,1)</code>
$d_i = m_{i+1} - m_i$	<code>d_a</code>
$C_i = S_{i,i+1} - S_i$	<code>C_a</code>

B V-Hutch restoration (opt-in)

Default production drops the V term entirely (§6). For problem classes where the geometric argument is suspect — boundary or cluster-supported coverage, anisotropic coverage, sharp turns

near $\partial\Omega$, cold-start outer iters before $q(x)$ has localised — this appendix gives a Hutchinson + NUFFT restoration whose (m_i, S_i) gradient structure is exactly the linear case of §6.

B.1 Closed-form $\mathbb{E}_{q(x_i)}[V]$ via BTTB-diagonal sums of $DA^{-1}D$

Define the diagonal sums

$$\omega_r(\delta) := \sum_{(k,\ell): \xi_\ell - \xi_k = \delta} D_k (A_r^{-1})_{k\ell} D_\ell, \quad (42)$$

the BTTB projection of $D_\theta A_r^{-1} D_\theta$ onto lags δ on the $2M$ -padded spectral-difference grid. By stationarity of the kernel, $\omega_r(-\delta) = \omega_r(\delta)^*$ (Hermitian-symmetric; generally complex when source locations break centrosymmetry). Then

$$\mathbb{E}_{q(x_i)}[V_r(x_i)] = \sum_{\delta} \omega_r(\delta) e^{2\pi i \delta^\top m_i} e^{-2\pi^2 \delta^\top S_i \delta}, \quad (43)$$

a Gaussian-enveloped Type-2 NUFFT against the difference-grid coefficient block — the primitive `gmix_E_full_Eff` already uses for $\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$, with ω_r replacing the autocorrelation ρ_r of μ_r .

B.2 Hutchinson estimator of ω_r

ω_r is estimated unbiasedly by a padded-FFT pair on P Rademacher probes $z_p \in \{\pm 1\}^M$:

$$\hat{\omega}_r = \frac{1}{P} \sum_{p=1}^P \mathcal{F}^{-1}(\overline{\mathcal{F}[Du_{p,r}]} \cdot \mathcal{F}[Dz_p]), \quad u_{p,r} = A_r^{-1} z_p, \quad (44)$$

with $A_r^{-1} z_p$ via the same Jacobi-preconditioned CG that drives the $q(f)$ update. Variance scales as $\|DA^{-1}D\|_F/\sqrt{P}$; $P = 4$ probes shared across the D_{out} output dims are sufficient. *Keep the imaginary part:* a `.real` cast on the IFFT discards $-\Im\omega(\delta) \sin(2\pi\delta^\top m)$, the dominant error vs dense ground truth when sources are not centrosymmetric.

B.3 Per-transition S_i -gradient via Taylor

The §6 linear-case identity (40) applies directly:

$$\mathbb{E}_{q(x_i)}[V(x_i)] = V(m_i), \quad \partial_{m_i} \mathbb{E}[V] = \nabla V(m_i), \quad \partial_{S_i} \mathbb{E}[V] = 0. \quad (45)$$

The `custom_vjp` shim exposes V_i and $\partial V_i / \partial m_i$ to `nat_grad_transition`'s autodiff and sets the $\partial V_i / \partial S_i$ slot to zero; this is the V-analog of the linearised- $\bar{f}$ shim in §6, applied at the same Taylor truncation order.

The dropped second-order Taylor term has bias $\mathbb{E}_{q(x_i)}[V(x_i)] - V(m_i) = \frac{1}{2} \text{tr}[\nabla^2 V(m_i) S_i] + O(\|S_i\|^2)$, bounded by $\|S_i\| \cdot \sigma_f^2 / \ell^2$ for SE kernels — small for $q(x)$ localised at scale $\sim \ell$, and contributes only to V_i 's constant offset in $\mathbb{E}[\bar{f}^\top \bar{f}]$, not to its (m, S) gradient. The exposed *value* V_i is the exact closed-form (43); the Taylor argument applies only to the *gradients*. To restore the second-order S -pull exactly, replace V by its second-order Taylor with PSD-projected Hessian $H_V^+ \succeq 0$ (Price's identity, $\partial_S = \frac{1}{2} H_V^+$); cost: $D(D+1)/2$ extra batched NUFFTs against frequency-modulated $\omega(\delta) \cdot (-2\pi^2 \delta_j \delta_k) \text{env}(\delta)$ coefficient grids. Not enabled by default.

B.4 Homogeneous- S vs heterogeneous- S sub-modes

Two production sub-modes for the per-source envelope in (43):

'hutch' (default). S_i is replaced by the trajectory-mean $\bar{S}$ in the envelope $e^{-2\pi^2\delta^\top S\delta}$, so the envelope folds into the coefficient grid once per outer iter and the per-source NUFFT-2 is a standard (non-gmix) transform.

'hutch_hetS'. Per-source S_i envelope, implemented via the Type-2 gmix-gather primitive (`sing/efgp_gmix_gather`). Pipeline: (i) inverse-FFT $\hat{\omega}(\delta)$ once to a uniform spatial grid $\tilde{\omega}(x)$; (ii) for each source i , gather $\tilde{\omega}$ via a per-source Gaussian stencil weighted by $\mathcal{N}(x; m_i, S_i)$ and Riemann-summed over a $(2r+1)^D$ neighbourhood, with $r = \lceil n_\sigma \sigma_{\max}/h_{\text{grid}} \rceil$. Both V_i and ∇V_i flow through the same primitive (gradient via `jax.grad` reverse-mode through the gather).

C Implementation and benchmarks

C.1 Custom-VJP shims: why a shim, not pure autodiff

`nat_grad_transition` runs `vmap(grad(loss_one_transition))` where `loss_one_transition` closes over a single (m_t, S_t) . Inside that `vmap`, JAX/XLA does *not* hoist the FFT across `vmap`'d `nufft2` calls: a per-source NUFFT-2 inside `ff(t, m, S)` pays the full $O(M \log M)$ FFT cost *per source*, giving $O(NM \log M)$ per inner iter and $\sim 50\times$ slowdown at $K=10/T=500$ (measured). `FrozenEFGPDrift` sidesteps this by precomputing $(\bar{f}, J_{\bar{f}})$ at all sources via ONE forward batched NUFFT-2 plus its AD-driven adjoint (D Type-1 NUFFT) *outside* the per-transition `vmap`, then injecting both as constants per-transition via `custom_vjp` with the §6 gradient table. The V-Hutch shim `FrozenEFGPDriftWithVHutch` follows the same pattern for $(V_i, \nabla V_i)$. A refactor of `nat_grad_transition` to compute the multi-transition ELBO in one shot and `jax.grad` it once over all (m_i, S_i) would recover the same asymptotic cost without shims.

C.2 Point-eval alternative: linearised-shim

The §6 default Gaussian-averages $(\bar{f}, J_{\bar{f}})$ under $q(x_i)$ via the gmix-gather primitive. A cheaper alternative, *linearised-shim*, uses point-eval $(\bar{f}(m_i), J_{\bar{f}}(m_i))$ directly (no Gaussian averaging): one batched Type-2 NUFFT on μ_r at the $\{m_i\}$ source set, with no gather. The Taylor-truncated gradient structure of §6 is identical — so both consume the same `FrozenEFGPDrift` custom-VJP — but the base values miss the first-order S_i -correction. Empirically (§C wall-time table) the two paths reach the same fixed point to 3–4 decimals and within $\sim 7\%$ wall, so gmix-batched is the production default but linearised-shim is retained for problems where the gather truncation is not negligible (small ℓ regime) or for debugging.

C.3 Wall-time comparisons ($K = 10, T = 500$)

Benchmarks against the `'gmix_batched'` baseline, subprocess-isolated timings, warm JIT:

- Base-point choice: `'gmix_batched'` reaches the same $(\hat{\ell}, \hat{\sigma}_f^2, \text{lat_rmse})$ fixed point as `'linearised_shim'` to 3–4 decimals, at wall within $\sim 7\%$ (`demons/bench_qx_moments_modes.py`).
- V-restoration overhead: `'hutch'` ~ 53.7 s warm vs `'none'` ~ 48.3 s ($\sim 11\%$); `'hutch_hetS'` ~ 64.8 s ($\sim 1.34\times$). Recovery is bit-identical across all three modes (`demons/bench_hutch_vs_none.py`).

C.4 ∇V magnitude diagnostic

Setup: $K=10$ trials, $x_0 \sim \mathcal{U}([-2, 2]^2)$, integrated for 8 s with $\sigma=0.4$, $\ell_{\text{true}}=0.6$. Latent paths cover $\Omega \approx [-3.5, 3]^2$ with $\sim 5,000$ sources, $|\Omega|/\ell^d \approx 120$; the §6-A isotropic-coverage argument applies with margin.

Direct measurement at a representative transition shows the wiring is correct (the `custom.vjp` bwd produces ∇V matching the precomputed `grad_V_const` to numerical precision) but the *magnitude* of ∇V is $\sim 5 \times 10^{-5}$ of the $\partial \text{Eff}/\partial m = 2J^\top \mathbb{E}[\tilde{f}]$ term per transition, translating to $\sim 0.16\%$ change in the full `compute_neg_CE_single` gradient direction. SING’s ρ -damping washes this out. The bit-identical recovery between ‘none’ and ‘hutch’ (final $\hat{\ell}=1.1337$, $\hat{\sigma}_f^2=1.3230$, latent RMSE=0.0646, all matching to 3–4 decimals across all three V-restoration modes) reflects the structural argument, not a wiring bug.

C.5 Op count for V-Hutch per outer iter

Operation	Count	Per-call cost
Hutch CG ($A_r^{-1}z_p$, eq. 44)	$P \cdot D_{\text{out}}$	$O(k_{\text{cg}} M \log M)$
FFT cross-correlation (gives $\hat{\omega}_r$)	$P \cdot D_{\text{out}}$	$O(M \log M)$
Forward batched NUFFT-2 (gives $\{V_i\}$)	1	$O(K(M \log M + T))$
Adjoint NUFFT-1 (∇V_i via <code>jax.grad</code>)	D	$O(K(M \log M + T))$
Per-inner-iter NUFFTs / CG solves added		0

NUFFTs are the only ops that scale with $N = K(T-1)$. Per-outer-iter counts: $1+D$ batched NUFFTs, $P \cdot D_{\text{out}}$ CG solves; both independent of n_{estep} . At $K=10/T=500$ ($D=2$, $M \sim 10^3$, $P=4$, $k_{\text{cg}} \sim 30$): CG block ~ 0.1 s, NUFFT block ~ 1 ms; total $\sim 5\text{--}10\%$ theoretical overhead, within JIT noise empirically.